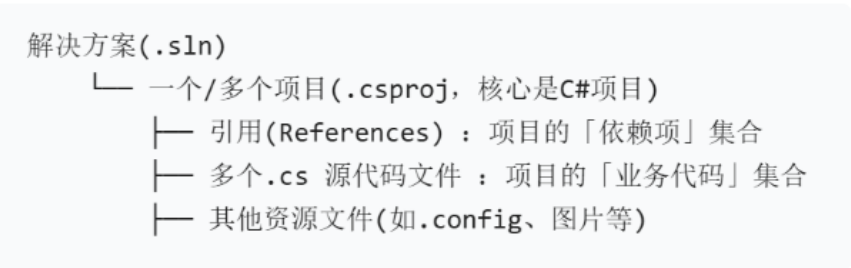
1.入门知识点补充

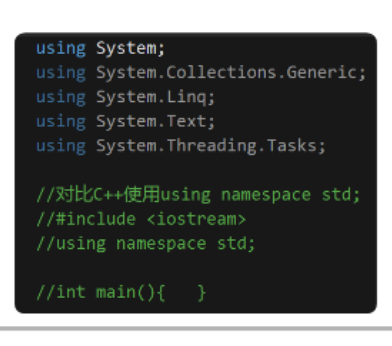
C#文件夹的组成



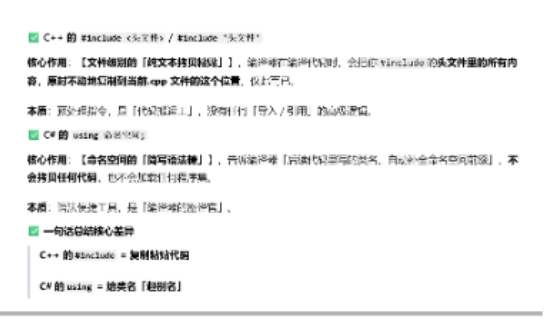
```
解决方案(.sln)
    └── 一个/多个项目(.csproj，核心是C#项目)
        ├── 引用(References) ：项目的「依赖项」集合
        ├── 多个.cs 源代码文件 ：项目的「业务代码」集合
        ├── 其他资源文件(如.config、图片等)
```

- 解决方案相当于项目的容器,一个项目最终会编译为exe

命名空间



```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

//对比C++使用using namespace std;
//#include <iostream>
//using namespace std;

//int main(){   }
```



✅ C++ 的 #include <头文件> / #include "头文件"

✅ C# 的 using 命名空间;

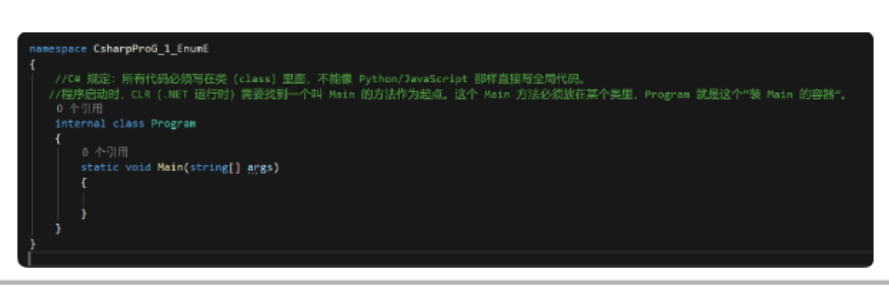
✅ 一句话总结核心差异

C++ 的 #include = 复制粘贴代码

C# 的 using = 给类名「起别名」

- using system导入命名空间而不是一开始就全部默认存在,就是为了防止不同命名空间下名字冲突
- 区分不同namespace下相同名称的class 类

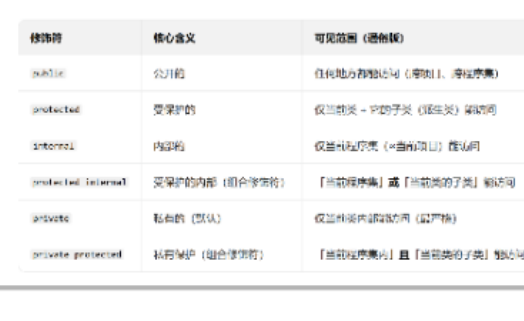
启动类



```
namespace CsharpProG_1_EnumE
{
    //C# 规定：所有代码必须写在类（class）里面，不能像 Python/JavaScript 那样直接写全局代码。
   //程序启动时，CLR（.NET 运行时）需要找到一个叫 Main 的方法作为起点。这个 Main 方法必须放在某个类里，Program 就是这个"装 Main 的容器"。
    0 个引用
    internal class Program
    {
        0 个引用
        static void Main(string[] args)
        {

        }
    }
}
```

主方法的格式

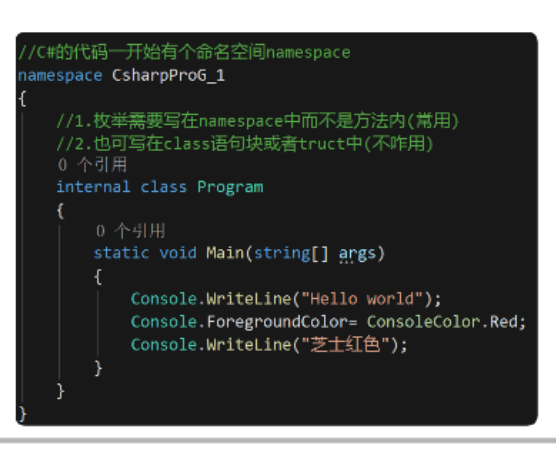
- namespace CsharpProG_1{}　　命名空间名字和解决方案名字不需要强制对应,但是一般命名空间和解决方案同名
- internal class Program{}
  - 一个项目编译后就是一个程序集(也就是一个exe/dll文件)访问,一一对应
  - interal和public　　其实就是要不要让挂载脚本的gameobject知道脚本数据



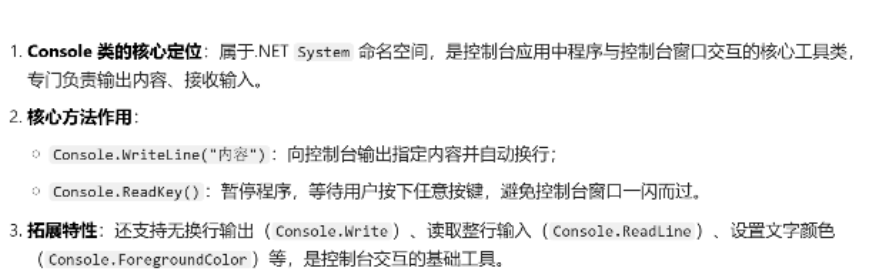
修饰符	核心含义	可见范围（通俗版）
public	公开的	任何地方都能访问（[illegible]）
protected	受保护的	仅当前类 + 它的子类（派生类）能访问
internal	内部的	仅当前程序集（=当前项目）能访问
protected internal	受保护的内部（组合修饰符）	「当前程序集」或「当前类的子类」能访问
private	私有的（默认）	仅当前类内部能访问（最严格）
private protected	私有保护（组合修饰符）	「当前程序集内」且「当前类的子类」能访问

- **static void Main(string[] args)**
  - **必须是静态的（static）**：因为程序启动时还没有创建任何类的实例，静态方法无需实例即可执行；
  - Java 的程序入口必须是 public static void main(String[] args)
    - public：必须加，否则 JVM 无法访问这个方法（C# 可省略）；
    - 方法名必须是小写 main，写成 Main 会被识别为普通方法，不是入口。
    - String[] args：参数名可以改（比如改成 String[] params）　　只是个参数名,写成X都可以
  - C++就是朴实无华的int main()
- **参数可选**：string[] args 用于接收命令行参数（比如运行程序时传入的参数）；
- **返回值可选**：void 无返回值，int 可返回退出码（0 表示正常，非 0 表示异常）；
- `一个程序只能有一个可执行的 Main 方法`：如果多个类有 Main，需要在项目属性中指定 "启动对象"。
- **启动方法**：侧重 "程序启动时第一个执行的方法" 这一特性；
- 如果改为 static int Main(string[] args)
  - return 0代表程序正常执行
  - return 1则是代表程序异常执行

console类



```
//C#的代码一开始有个命名空间namespace
namespace CsharpProG_1
{
    //1.枚举需要写在namespace中而不是方法内(常用)
    //2.也可写在class语句块或者truct中(不咋用)
    0 个引用
    internal class Program
    {
        0 个引用
        static void Main(string[] args)
        {
            Console.WriteLine("Hello world");
            Console.ForegroundColor= ConsoleColor.Red;
            Console.WriteLine("芝士红色");
        }
    }
}
```

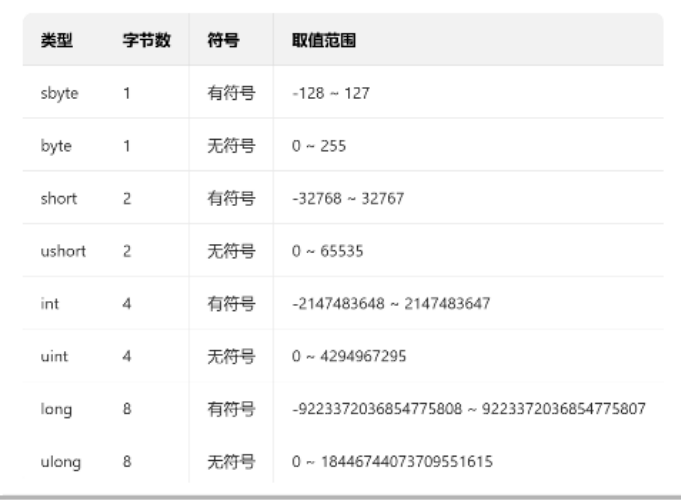


1. **Console 类的核心定位**：属于.NET `System` 命名空间，是控制台应用中程序与控制台窗口交互的核心工具类，专门负责输出内容、接收输入。
2. **核心方法作用**：
   - `Console.WriteLine("内容")`：向控制台输出指定内容并自动换行；
   - `Console.ReadKey()`：暂停程序，等待用户按下任意按键，避免控制台窗口一闪而过。
3. **拓展特性**：还支持无换行输出（`Console.Write`）、读取整行输入（`Console.ReadLine`）、设置文字颜色（`Console.ForegroundColor`）等，是控制台交互的基础工具。

直接启动和调试启动

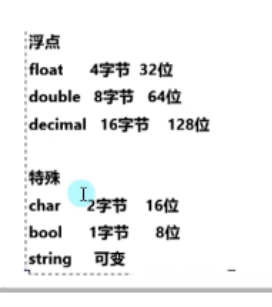
- 直接启动就是速度更快,但是报错了没法定位到错误位置只是终止运行

占用字节数



类型	字节数	符号	取值范围
sbyte	1	有符号	-128 ~ 127
byte	1	无符号	0 ~ 255
short	2	有符号	-32768 ~ 32767
ushort	2	无符号	0 ~ 65535
int	4	有符号	-2147483648 ~ 2147483647
uint	4	无符号	0 ~ 4294967295
long	8	有符号	-9223372036854775808 ~ 9223372036854775807
ulong	8	无符号	0 ~ 18446744073709551615

- **sbyte**：全称是 signed byte（有符号字节）
- sbyte是10000000到01111111　　补码表示



浮点
float　4字节　32位
double　8字节　64位
decimal　16字节　128位

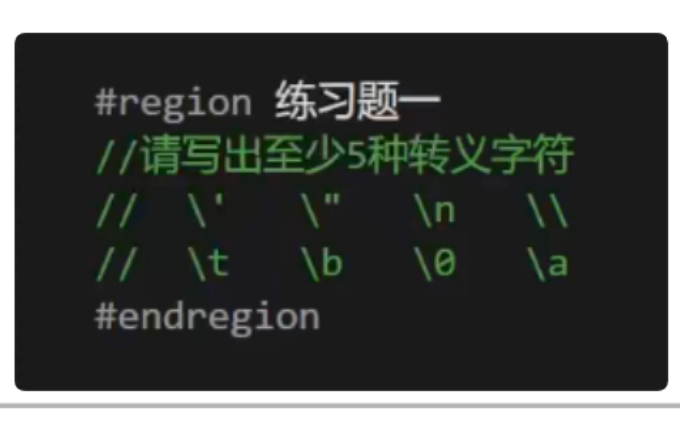
特殊
char　2字节　16位
bool　1字节　8位
string　可变

- 字符串字节数可变

常量

- const int x=99;
  - 一定有初始值(初始化)
  - 不可被修改

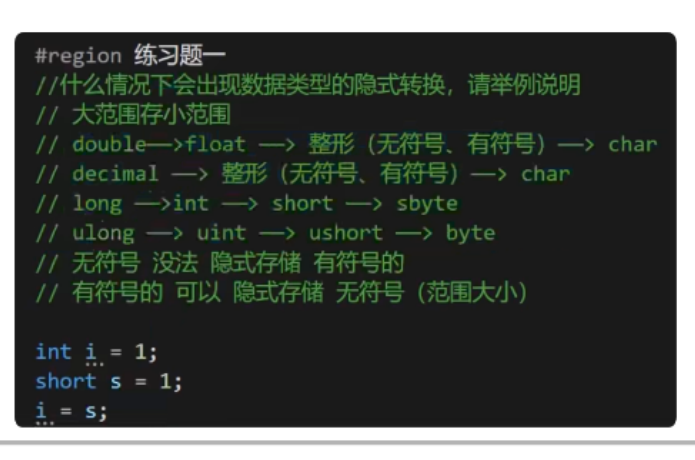
转义字符



```
#region 练习题一
//请写出至少5种转义字符
//  \'   \"   \n   \\
//  \t   \b   \0   \a
#endregion
```

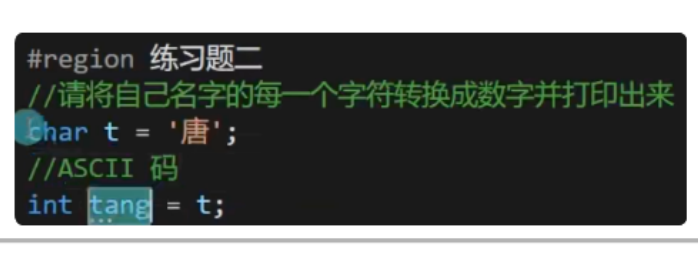
- \'和\"就是用于在字符/字符串中插入" "和' '
- \a是终端提示音,反正插入字符串就会叮咚的叫一声

隐式转换

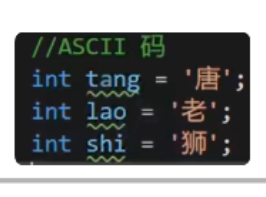


```
#region 练习题一
//什么情况下会出现数据类型的隐式转换，请举例说明
// 大范围存小范围
// double—>float —> 整形（无符号、有符号）—> char
// decimal —> 整形（无符号、有符号）—> char
// long —>int —> short —> sbyte
// ulong —> uint —> ushort —> byte
// 无符号 没法 隐式存储 有符号的
// 有符号的 可以 隐式存储 无符号（范围大小）

int i = 1;
short s = 1;
i = s;
```

- 隐式转换:自动小变大　　为了防止数据丢失,大变小必须强制转换
- "",扩大范围就是合理扩充二进制位,当然没问题了　　int+char=int
- 输入文字
- 只有极少数并不严谨的语言才能使用大变小的隐式转化(仍旧是不安全行为)



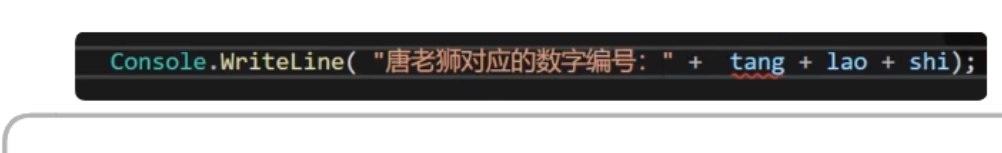
```
#region 练习题二
//请将自己名字的每一个字符转换成数字并打印出来
char t = '唐';
//ASCII 码
int tang = t;
```



```
//ASCII 码
int tang = '唐';
int lao = '老';
int shi = '狮';
```

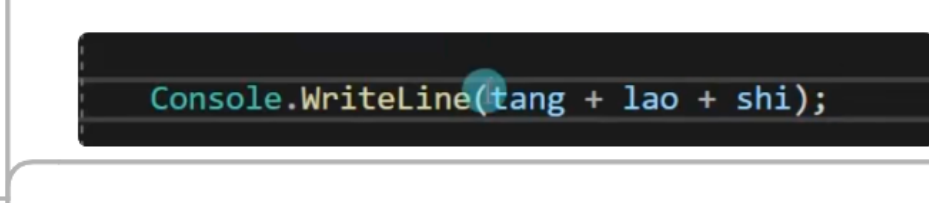
字符串拼接

加法拼接



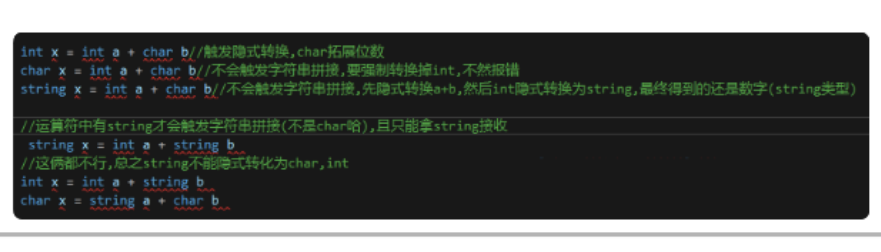
```
Console.WriteLine( "唐老狮对应的数字编号：" +  tang + lao + shi);
```

- 加上字符串整段话才是打印字符



```
Console.WriteLine(tang + lao + shi);
```

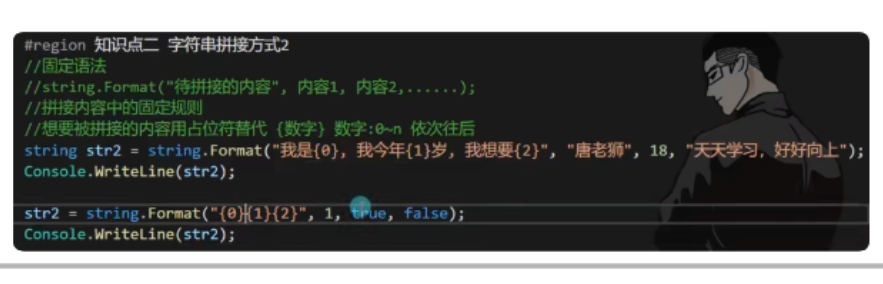
- 直接打印得出的是ASCII码相加的结果



```
int x = int a + char b//触发隐式转换,char拓展位数
char x = int a + char b//不会触发字符串拼接,要强制转换掉int,不然报错
string x = int a + char b//不会触发字符串拼接,先隐式转换a+b,然后int隐式转换为string,最终得到的还是数字(string类型)

//运算符中有string才会触发字符串拼接(不是char哈),且只能拿string接收
 string x = int a + string b
//这俩都不行,总之string不能隐式转化为char,int
int x = int a + string b
char x = string a + char b
```

- string是引用类型,int和char是值类型　　隐式转化和强制转化(括号法)都不行
- 但是可以用Parse和Convert转化
  - int num1 = int.Parse(numStr);
  - int num2 = Convert.ToInt32(nullStr);

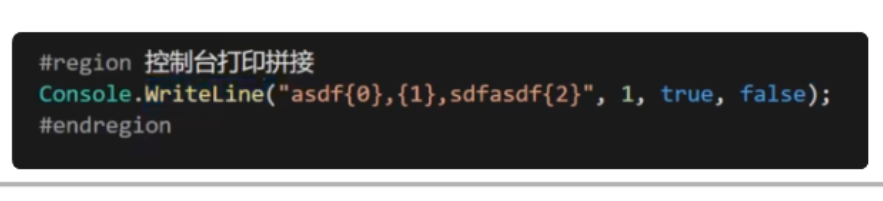
Format方法拼接



```
#region 知识点二 字符串拼接方式2
//固定语法
//string.Format("待拼接的内容", 内容1, 内容2,......);
//拼接内容中的固定规则
//想要被拼接的内容用占位符替代 {数字} 数字:0~n 依次往后
string str2 = string.Format("我是{0}, 我今年{1}岁, 我想要{2}", "唐老狮", 18, "天天学习, 好好向上");
Console.WriteLine(str2);

str2 = string.Format("{0}{1}{2}", 1, true, false);
Console.WriteLine(str2);
```

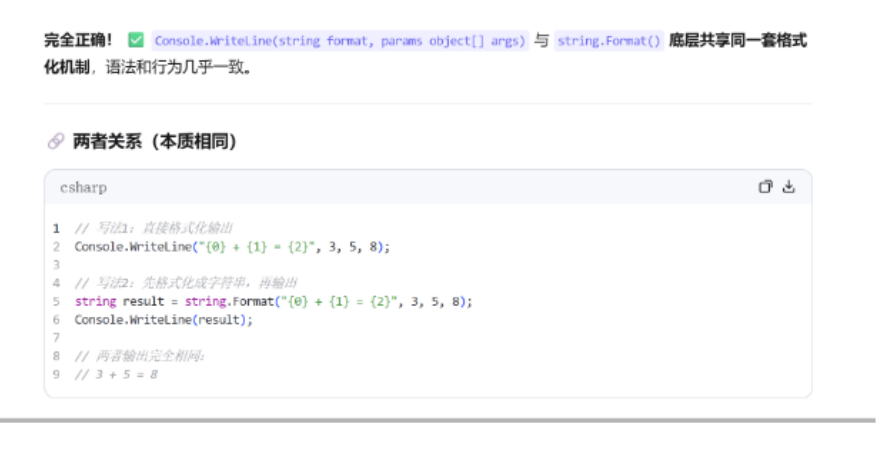
- string.Format()
- 第一个参数是主句,后面的依次是待替换进去的内容　　比如第二句就是打印的12truefalse
- 这段话赋值给str2

控制台打印



```
#region 控制台打印拼接
Console.WriteLine("asdf{0},{1},sdfasdf{2}", 1, true, false);
#endregion
```

- 控制台默认就提供了string.Format
- console.writeline()方法中有多个参数的时候,不是打印多个,而是打印拼接

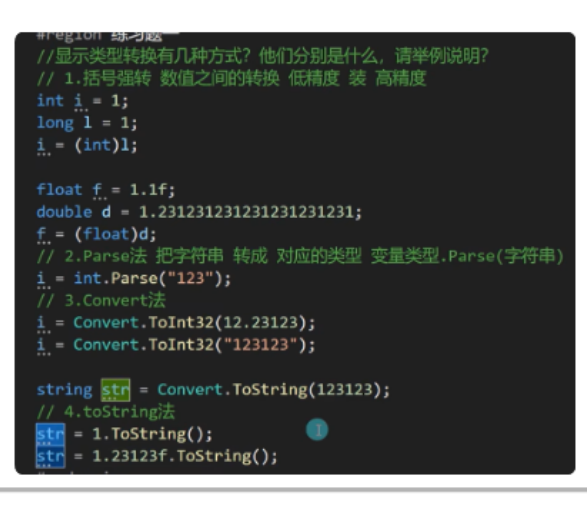


完全正确！ ✅ `Console.WriteLine(string format, params object[] args)` 与 `string.Format()` **底层共享同一套格式化机制**，语法和行为几乎一致。

两者关系（本质相同）

```csharp
// 写法1：直接格式化输出
Console.WriteLine("{0} + {1} = {2}", 3, 5, 8);

// 写法2：先格式化成字符串，再输出
string result = string.Format("{0} + {1} = {2}", 3, 5, 8);
Console.WriteLine(result);

// 两者输出完全相同：
// 3 + 5 = 8
```

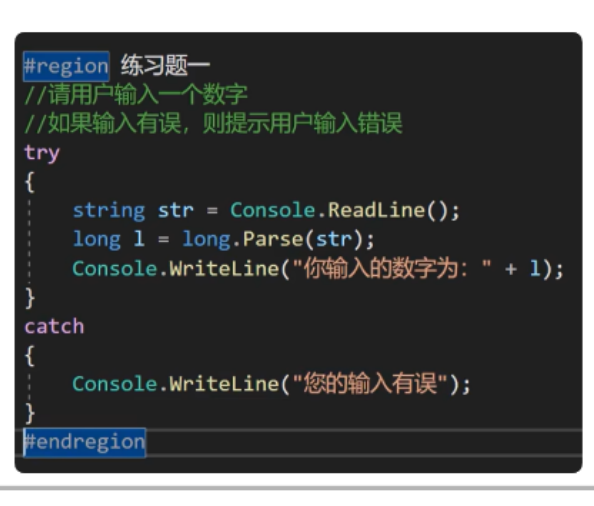
显式转换



```
//显示类型转换有几种方式? 他们分别是什么，请举例说明?
// 1.括号强转 数值之间的转换 低精度 装 高精度
int i = 1;
long l = 1;
i = (int)l;

float f = 1.1f;
double d = 1.2312312312312312312311;
f = (float)d;
// 2.Parse法 把字符串 转成 对应的类型 变量类型.Parse(字符串)
i = int.Parse("123");
// 3.Convert法
i = Convert.ToInt32(12.23123);
i = Convert.ToInt32("123123");

string str = Convert.ToString(123123);
// 4.toString法
str = 1.ToString();
str = 1.23123f.ToString();
```

- Convert任意类型转为任意类型
  - Convert.ToString()是唯一专门转化为String的方法
  - 当然Convert本身还是很多用的
  - Convert.　　每种方法不支持的参数都不同,这个我感觉只能现查了
- Parse字符串转化为任意类型
  - Parse是专门将string转化为其他类型的变量
  - 参数只支持纯数字类型的字符串　　XXX.Parse()就是将不同的纯数字字符串转为对应类型变量
- 对字符串不能直接转化为对应的ascii码,没法直接(int)numstr;
  - 要么int.Parse()
  - 要么Convert.ToInt32()

异常捕获

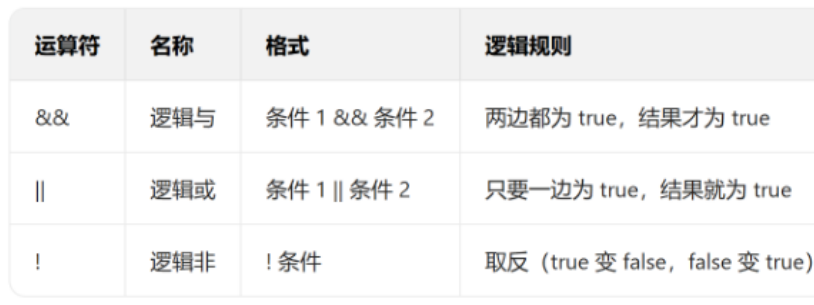


```
#region 练习题一
//请用户输入一个数字
//如果输入有误，则提示用户输入错误
try
{
    string str = Console.ReadLine();
    long l = long.Parse(str);
    Console.WriteLine("你输入的数字为：" + l);
}
catch
{
    Console.WriteLine("您的输入有误");
}
#endregion
```

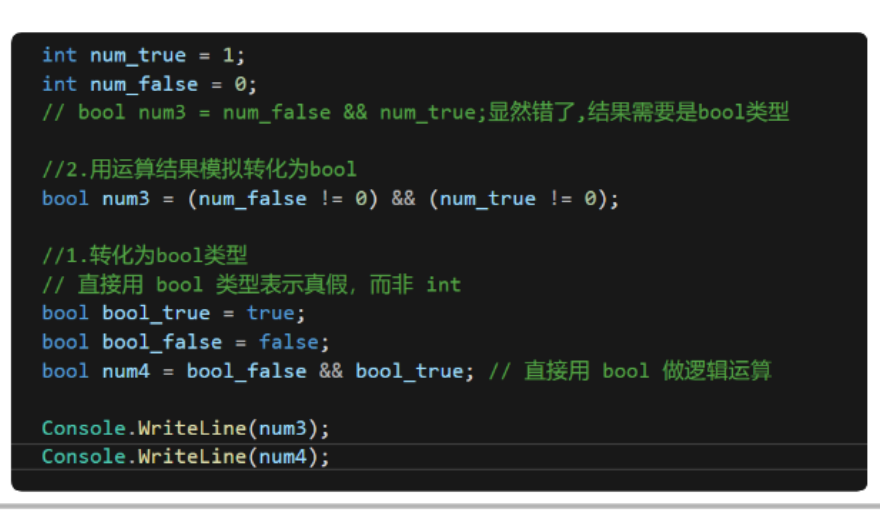
- try{}中输入错误,就执行catch{}
- 用str接收Console.ReadLine()用户输入

运算符

逻辑运算符

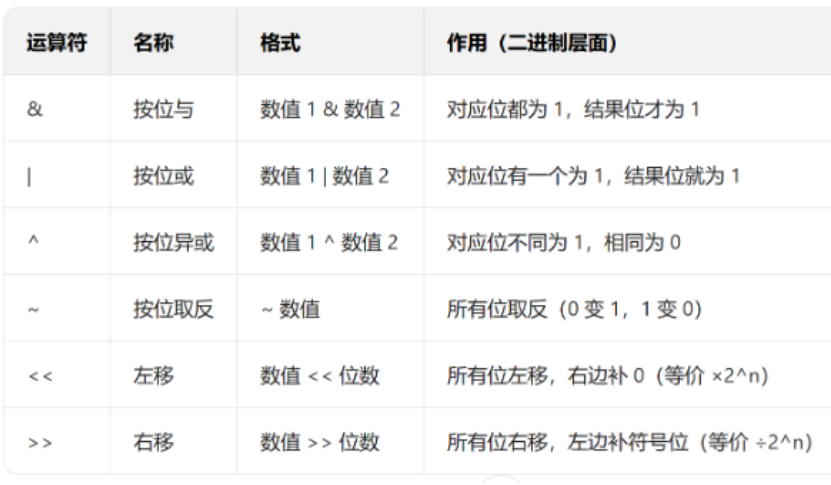


运算符	名称	格式	逻辑规则
&&	逻辑与	条件 1 && 条件 2	两边都为 true，结果才为 true
||	逻辑或	条件 1 || 条件 2	只要一边为 true，结果就为 true
!	逻辑非	! 条件	取反（true 变 false，false 变 true）

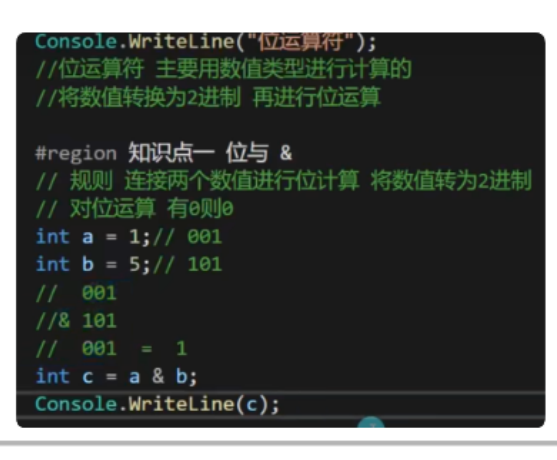


```
int num_true = 1;
int num_false = 0;
// bool num3 = num_false && num_true;显然错了,结果需要是bool类型

//2.用运算结果模拟转化为bool
bool num3 = (num_false != 0) && (num_true != 0);

//1.转化为bool类型
// 直接用 bool 类型表示真假，而非 int
bool bool_true = true;
bool bool_false = false;
bool num4 = bool_false && bool_true; // 直接用 bool 做逻辑运算

Console.WriteLine(num3);
Console.WriteLine(num4);
```

位运算符

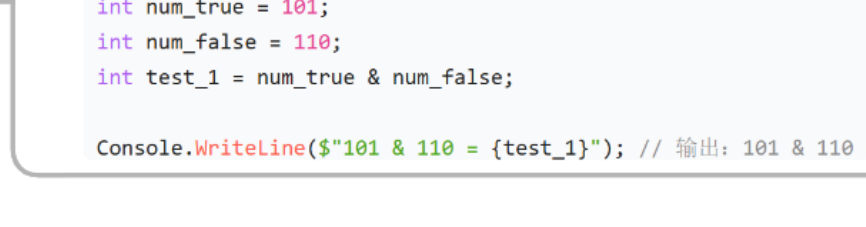


运算符	名称	格式	作用（二进制层面）
&	按位与	数值 1 & 数值 2	对应位都为 1，结果位才为 1
|	按位或	数值 1 | 数值 2	对应位有一个为 1，结果位就为 1
^	按位异或	数值 1 ^ 数值 2	对应位不同为 1，相同为 0
~	按位取反	~ 数值	所有位取反（0 变 1，1 变 0）
<<	左移	数值 << 位数	所有位左移，右边补 0（等价 ×2^n）
>>	右移	数值 >> 位数	所有位右移，左边补符号位（等价 ÷2^n）



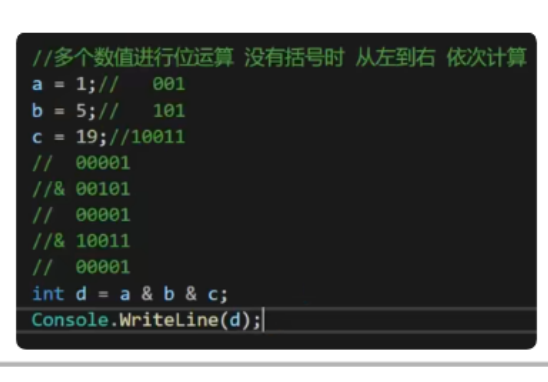
```
Console.WriteLine("位运算符");
//位运算符 主要用数值类型进行计算的
//将数值转换为2进制 再进行位运算

#region 知识点一 位与 &
// 规则 连接两个数值进行位计算 将数值转为2进制
// 对位运算 有0则0
int a = 1;// 001
int b = 5;// 101
//  001
//& 101
//  001  =  1
int c = a & b;
Console.WriteLine(c);
```

- 其实就是转为二进制,然后每位与运算
- 一定要有一个变量来接受结果

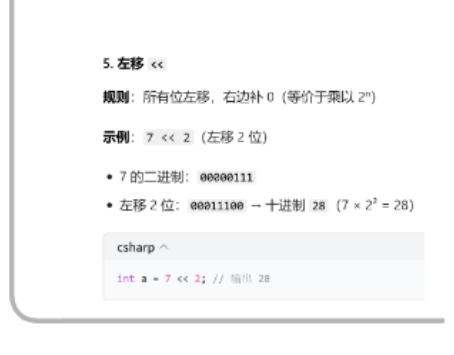


```
int num_true = 101;
int num_false = 110;
int test_1 = num_true & num_false;

Console.WriteLine($"101 & 110 = {test_1}"); // 输出: 101 & 110 = 100
```

- 这里两个运算数都是转化为对应的二进制再计算的



```
//多个数值进行位运算 没有括号时 从左到右 依次计算
a = 1;//   001
b = 5;//   101
c = 19;//10011
//  00001
//& 00101
//  00001
//& 10011
//  00001
int d = a & b & c;
Console.WriteLine(d);
```



5. 左移 `<<`

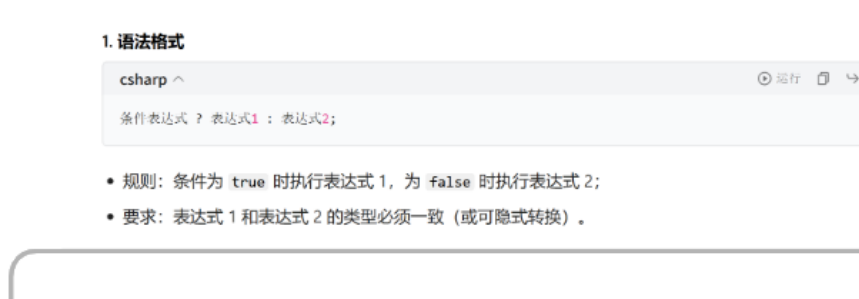
规则：所有位左移，右边补 0（等价于乘以 2^n）

示例：`7 << 2`（左移 2 位）

- 7 的二进制：`00000111`
- 左移 2 位：`00011100` → 十进制 `28`（$7 \times 2^2 = 28$）

```csharp
int a = 7 << 2; // 输出 28
```

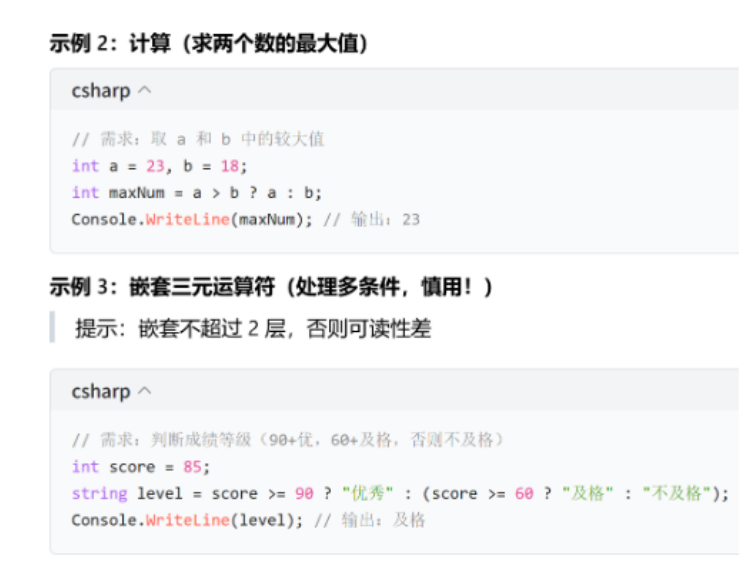
三目/条件 运算符



1. 语法格式

```csharp
条件表达式 ? 表达式1 : 表达式2;
```

- 规则：条件为 `true` 时执行表达式 1，为 `false` 时执行表达式 2；
- 要求：表达式 1 和表达式 2 的类型必须一致（或可隐式转换）。



示例 2：计算（求两个数的最大值）

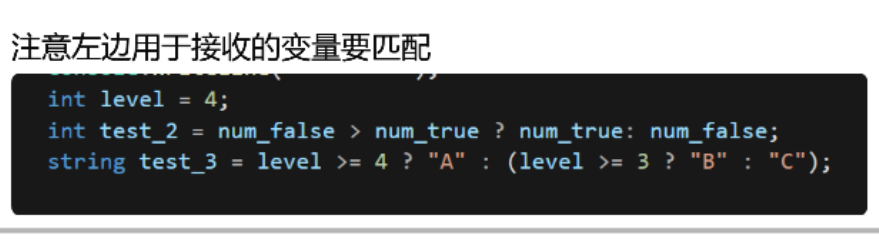
```csharp
// 需求：取 a 和 b 中的较大值
int a = 23, b = 18;
int maxNum = a > b ? a : b;
Console.WriteLine(maxNum); // 输出：23
```

示例 3：嵌套三元运算符（处理多条件，慎用！）

> 提示：嵌套不超过 2 层，否则可读性差

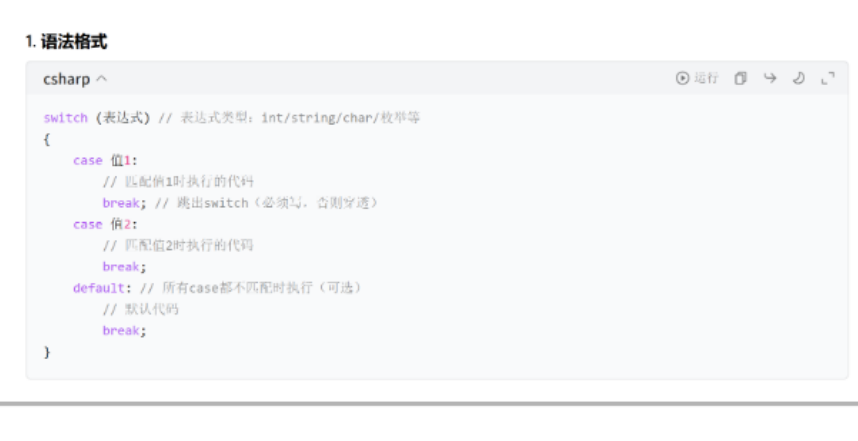
```csharp
// 需求：判断成绩等级（90+优，60+及格，否则不及格）
int score = 85;
string level = score >= 90 ? "优秀" : (score >= 60 ? "及格" : "不及格");
Console.WriteLine(level); // 输出：及格
```

- 注意左边用于接收的变量要匹配



```
int level = 4;
int test_2 = num_false > num_true ? num_true: num_false;
string test_3 = level >= 4 ? "A" : (level >= 3 ? "B" : "C");
```

switch语句



1. 语法格式

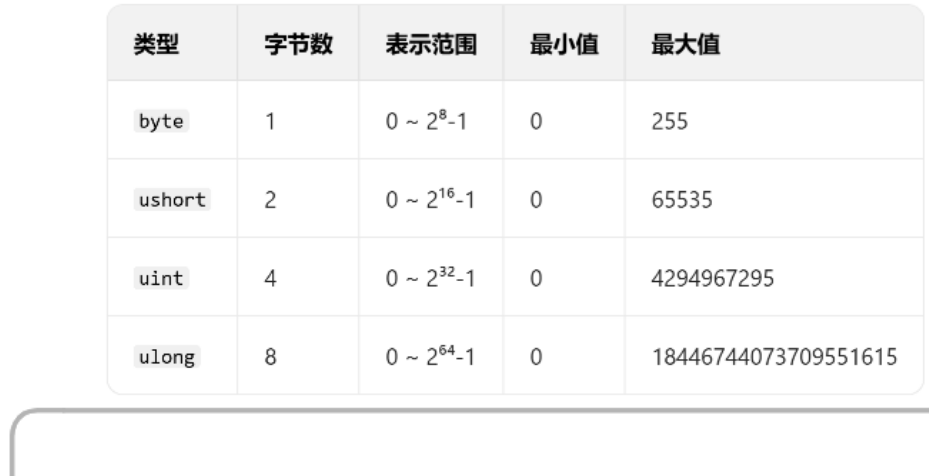
```csharp
switch (表达式) // 表达式类型：int/string/char/枚举等
{
    case 值1:
        // 匹配值1时执行的代码
        break; // 跳出switch（必须写，否则穿透）
    case 值2:
        // 匹配值2时执行的代码
        break;
    default: // 所有case都不匹配时执行（可选）
        // 默认代码
        break;
}
```

- **break 必须写**：不写会触发 "case 穿透"（执行当前 case 后继续执行下一个 case）；

do while语句

- 输入文字

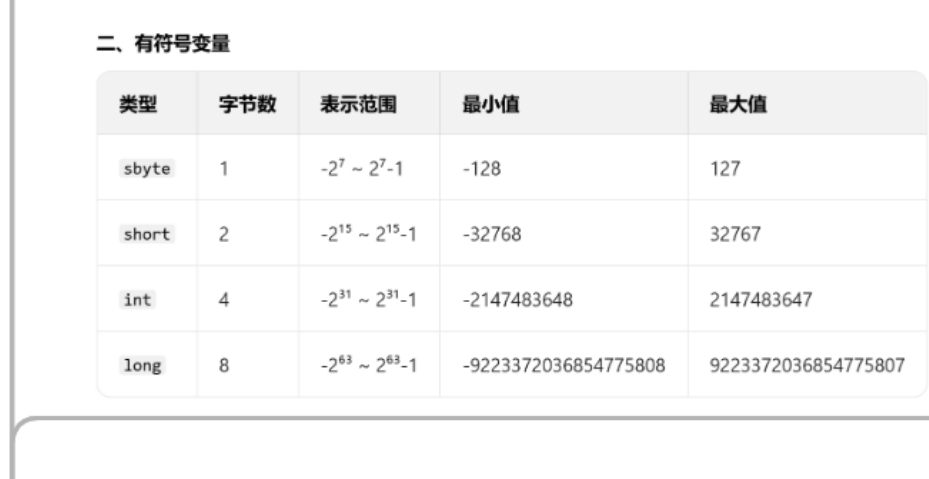
变量表示范围



一、无符号变量

类型	字节数	表示范围	最小值	最大值
`byte`	1	0 ~ 2^8-1	0	255
`ushort`	2	0 ~ 2^{16}-1	0	65535
`uint`	4	0 ~ 2^{32}-1	0	4294967295
`ulong`	8	0 ~ 2^{64}-1	0	18446744073709551615

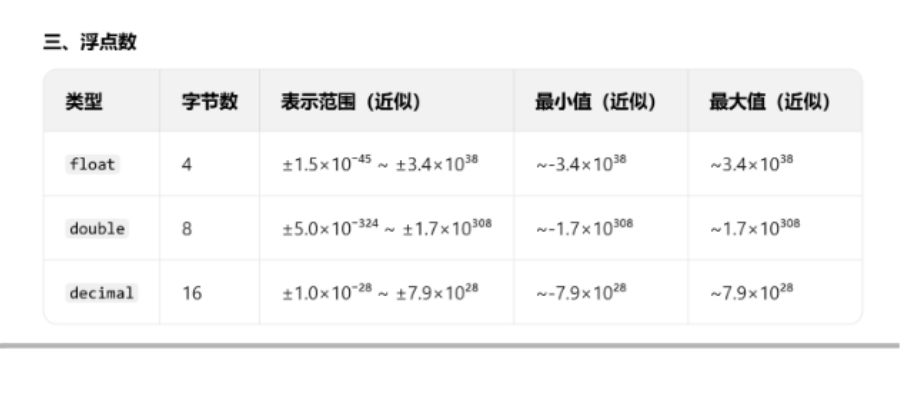
- 对于byte(注意是变量类型和二进制位不同),字节数为1,则有8bit,表示范围最大8次方
- int>short>byte是对的



二、有符号变量

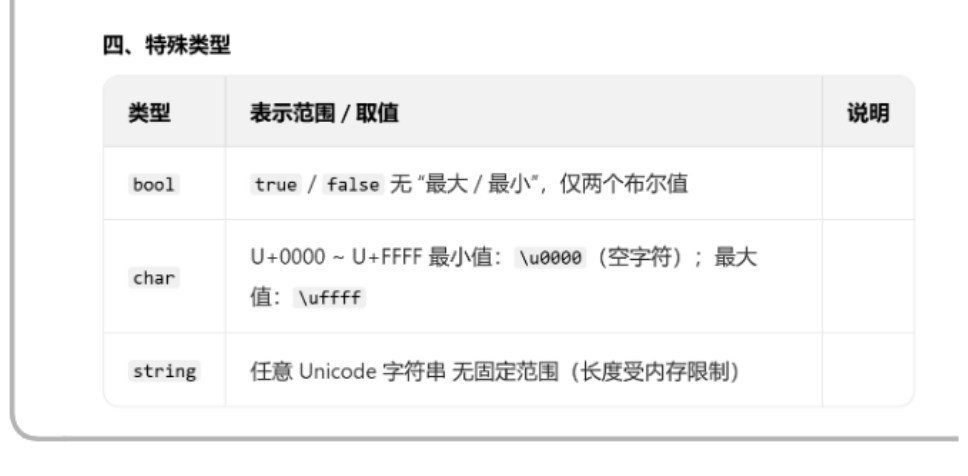
类型	字节数	表示范围	最小值	最大值
`sbyte`	1	-2^7 ~ 2^7-1	-128	127
`short`	2	-2^{15} ~ 2^{15}-1	-32768	32767
`int`	4	-2^{31} ~ 2^{31}-1	-2147483648	2147483647
`long`	8	-2^{63} ~ 2^{63}-1	-9223372036854775808	9223372036854775807

- byte无符号整数,sbyte则是有符号
- 有符号变量空出一位作为符号位,所以最大值小一个次方



三、浮点数

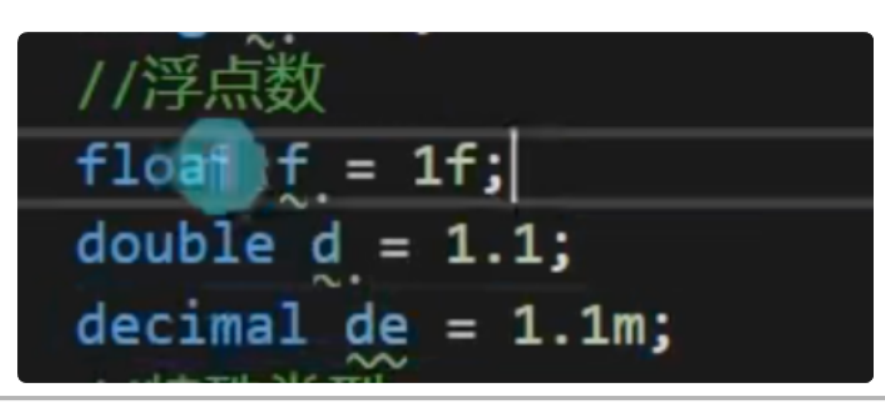
类型	字节数	表示范围（近似）	最小值（近似）	最大值（近似）
`float`	4	$\pm1.5\times10^{-45}$ ~ $\pm3.4\times10^{38}$	~-3.4×10^{38}	~3.4×10^{38}
`double`	8	$\pm5.0\times10^{-324}$ ~ $\pm1.7\times10^{308}$	~-1.7×10^{308}	~1.7×10^{308}
`decimal`	16	$\pm1.0\times10^{-28}$ ~ $\pm7.9\times10^{28}$	~-7.9×10^{28}	~7.9×10^{28}



四、特殊类型

类型	表示范围 / 取值	说明
`bool`	`true` / `false` 无 "最大 / 最小"，仅两个布尔值	
`char`	U+0000 ~ U+FFFF 最小值：`\u0000`（空字符）；最大值：`\uffff`	
`string`	任意 Unicode 字符串 无固定范围（长度受内存限制）	

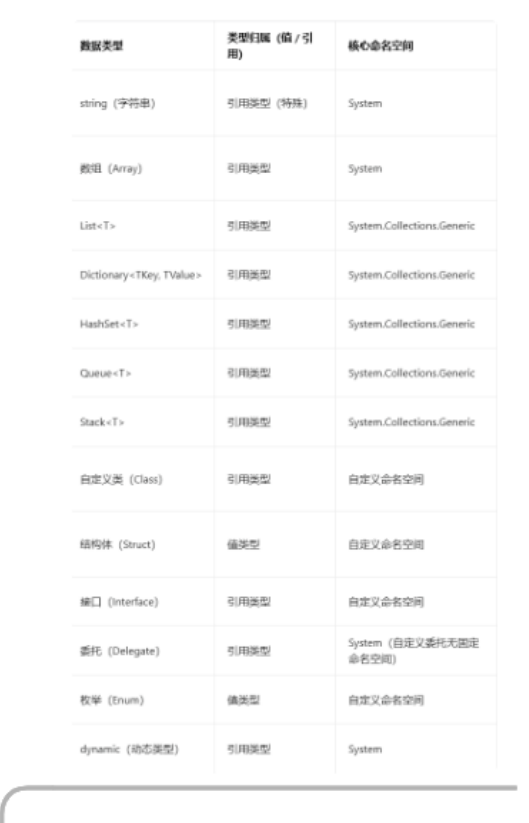
浮点数



```
//浮点数
float f = 1f;
double d = 1.1;
decimal de = 1.1m;
```

- 如果没有f,小数默认为double类型,此时赋值给float类型会报错

复杂数据类型



数据类型	类型归属（值 / 引用）	核心命名空间
string（字符串）	引用类型（特殊）	System
数组（Array）	引用类型	System
List<T>	引用类型	System.Collections.Generic
Dictionary<TKey, TValue>	引用类型	System.Collections.Generic
HashSet<T>	引用类型	System.Collections.Generic
Queue<T>	引用类型	System.Collections.Generic
Stack<T>	引用类型	System.Collections.Generic
自定义类（Class）	引用类型	自定义命名空间
结构体（Struct）	值类型	自定义命名空间
接口（Interface）	引用类型	自定义命名空间
委托（Delegate）	引用类型	System（自定义委托无固定命名空间）
枚举（Enum）	值类型	自定义命名空间
dynamic（动态类型）	引用类型	System

- 简单数据类型就是int,bool那些

by 狗大王 由 幕布 发布